Universidad Veracruzana
Desarrollo de Sistemas Web (DSW-19559)

P04. Arquitectura Web 3.0 con Angular, Spring Boot Java 11 y PostgreSQL

Tercer incremento del proyecto PR09
Monitoreo de huerto o ambiente

Equipo DSW-E01
Alejandro Pacheco Luna
Juan Pablo Kuri Ricardez
Ariadna Trejo Álvarez
Pedro García Padilla

Facilitador: Dr. Gabriel Rodríguez Vásquez
Repositorio: github.com/zywoxxx/dsw-p02-monitoreo-huerto
Orizaba, Ver., 9 de octubre de 2026

Contenido

1. Qué entregamos
2. De dónde partimos y qué conservamos
3. Componentes y responsabilidades
4. El flujo principal de punta a punta
5. Por qué Angular y Spring Boot: decisiones y alternativas
6. Modelo de datos en PostgreSQL
7. Endpoints y contratos
8. Seguridad básica
9. Despliegue y configuración
10. La estructura compila y corre
11. Trazabilidad RF → componente
12. Pruebas y evidencia
13. Problemas que tuvimos
14. Relación con la rúbrica R04
15. Aportación individual
16. Limitaciones y pendientes
Anexo. Comandos

1. Qué entregamos

Este documento acompaña al repositorio del equipo y describe el tercer incremento del PR09. En P02 el flujo principal corría con JSP y Servlets; en P03 lo pasamos a JSF con PrimeFaces. En P04 ya no se trata de cambiar cómo se pintan las pantallas, sino de separar la aplicación en dos piezas que se hablan por HTTP con JSON: un cliente Angular que corre en el navegador y una API REST hecha con Spring Boot sobre Java 11. La base de datos PostgreSQL es la misma de los dos incrementos anteriores y no cambió ni una columna.

La consigna pedía diseñar la arquitectura, documentar componentes, contratos, entidades, seguridad y despliegue, y entregar una estructura inicial compilable con trazabilidad RF-componente. Entregamos eso y, además, el flujo principal funcionando de punta a punta, porque era la única forma de comprobar que el contrato que escribimos se puede cumplir. Lo que no implementamos está marcado como pendiente, no escondido.

Elemento	Dónde está
Documento de arquitectura con diagramas y decisiones justificadas	docs/p04/arquitectura.md y docs/p04/diagramas/
Modelo PostgreSQL y su relación con los recursos de la API	docs/p04/modelo-datos.md, sql/01_schema.sql
Contrato OpenAPI 3.0.3 y catálogo de endpoints	docs/p04/openapi.yaml, docs/p04/endpoints.md
Esqueleto del backend Spring Boot (Java 11)	api-spring/
Esqueleto del frontend Angular 16	frontend-angular/
Trazabilidad RF → componente → prueba	docs/p04/trazabilidad-rf.md
Verificación, estados y limitaciones	docs/p04/verificacion.md
Evidencia (salidas de comandos y capturas) e índice frente a R04	docs/p04/evidencia/
Registro individual de aportación y decisiones	docs/p04/individual/ (entrega aparte: P04_APELLIDO_NOMBRE)
Bitácora con los fallos que encontramos y cómo los corregimos	docs/bitacora.md, entradas 25 a 33

2. De dónde partimos y qué conservamos

El commit de partida es bf78beb, el cierre de P03. De ahí tomamos sin cambios la base de datos (siete tablas), los datos ficticios, los tres roles con sus permisos y las reglas del huerto: sensor existente y activo, valor numérico con máximo dos decimales y sin redondeo, rechazo de lo que sale del rango físico, alerta BAJA o ALTA cuando el valor sale del umbral operativo con los límites incluidos, y lectura más alerta en una sola transacción. El SQL de los repositorios también viene de P02; solo cambió la forma de ejecutarlo.

Las aplicaciones de P02 y P03 siguen en el repositorio y siguen funcionando contra la misma base. No las tocamos. La API usa el puerto 8081 justamente para no chocar con el Tomcat del 8080.

No agregamos funciones que el acta no tenga. Zonas, alertas y anotaciones existen en la base y en el contrato, pero en este incremento no tienen código; el simulador IoT corresponde a Web 4.0.

3. Componentes y responsabilidades

La figura 1 muestra las tres capas y lo que hay dentro de cada una. Lo que está en verde ya corre; la caja gris de abajo lista lo que solo existe en el contrato.

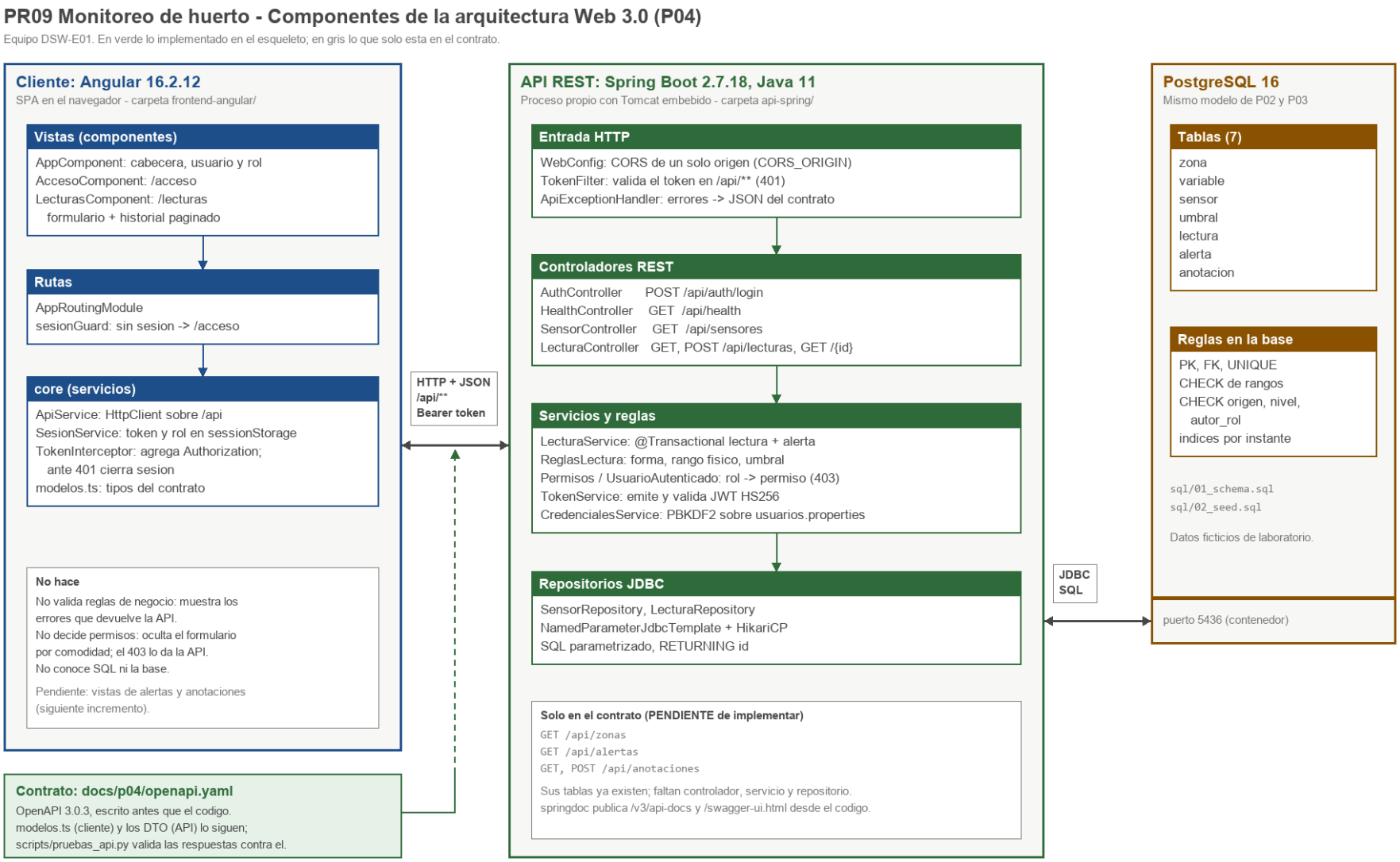


Figura 1. Diagrama de componentes: cliente Angular, API Spring Boot y PostgreSQL.

3.1 Cliente Angular

Componente	Qué hace	Qué no hace
AppComponent	Cabecera con usuario y rol, botón de salir, navegación	—
AccesoComponent (/acceso)	Pide usuario y contraseña, llama a POST /api/auth/login y abre la sesión	No compara contraseñas; no conoce la lista de usuarios
LecturasComponent (/lecturas)	Formulario de captura, historial paginado con filtro por procedencia, mensajes de éxito, alerta y error	No decide si un valor es válido: envía lo capturado y muestra lo que responde la API
sesionGuard	Impide entrar a /lecturas sin sesión	No es la seguridad real; la API responde 401 de todos modos
ApiService	Único lugar que conoce las rutas de la API	—
SesionService	Guarda token, usuario, rol y caducidad en sessionStorage; descarta sesiones vencidas	No guarda la contraseña
TokenInterceptor	Agrega Authorization: Bearer a cada llamada; ante un 401 cierra la sesión y regresa al acceso	—

3.2 API Spring Boot

Componente	Qué hace
WebConfig	CORS para /api/** con un solo origen permitido; metadatos de OpenAPI
TokenFilter	Exige token válido en /api/**, salvo /api/health, /api/auth/login y las peticiones OPTIONS; responde 401 con el cuerpo de error del contrato
AuthController	POST /api/auth/login: valida que vengan los dos campos (400), comprueba credenciales (401) y emite el token
HealthController	GET /api/health: 200 si llega a PostgreSQL, 503 si no
SensorController, SensorRepository	GET /api/sensores: catálogo con zona, variable, unidad, rango físico y umbral
LecturaController	GET y POST /api/lecturas, GET /api/lecturas/{id}; exige el permiso antes de llamar al servicio
LecturaService	Orquesta el registro: forma, existencia y estado del sensor, rango físico, inserción de lectura y alerta en una transacción
ReglasLectura	Las reglas puras, sin Spring ni base: obligatorios, dos decimales, rango físico, nivel y texto de la alerta
Permisos, UsuarioAutenticado	Tabla rol → permisos tomada del acta; exigir() lanza 403
TokenService, CredencialesService	Emite y valida el token (JWT HS256); compara la contraseña contra el hash PBKDF2 de usuarios.properties
LecturaRepository	SQL parametrizado con NamedParameterJdbcTemplate; no tiene reglas
ApiExceptionHandler	Traduce cualquier excepción al cuerpo Error del contrato; nunca manda trazas

3.3 PostgreSQL

Guarda los datos y hace cumplir lo que no debe depender del código: claves, unicidad, rangos (CHECK), valores permitidos de origen, nivel y autor_rol. Como tres aplicaciones escriben en las mismas tablas, estas restricciones son las que garantizan que un dato quede bien aunque entre por otro camino.

4. El flujo principal de punta a punta

El observador elige un sensor, escribe un valor y una observación, y el sistema guarda la lectura y, si hace falta, la alerta. La figura 2 sigue esa petición desde el navegador hasta PostgreSQL y marca en rojo dónde se corta cada caso inválido.

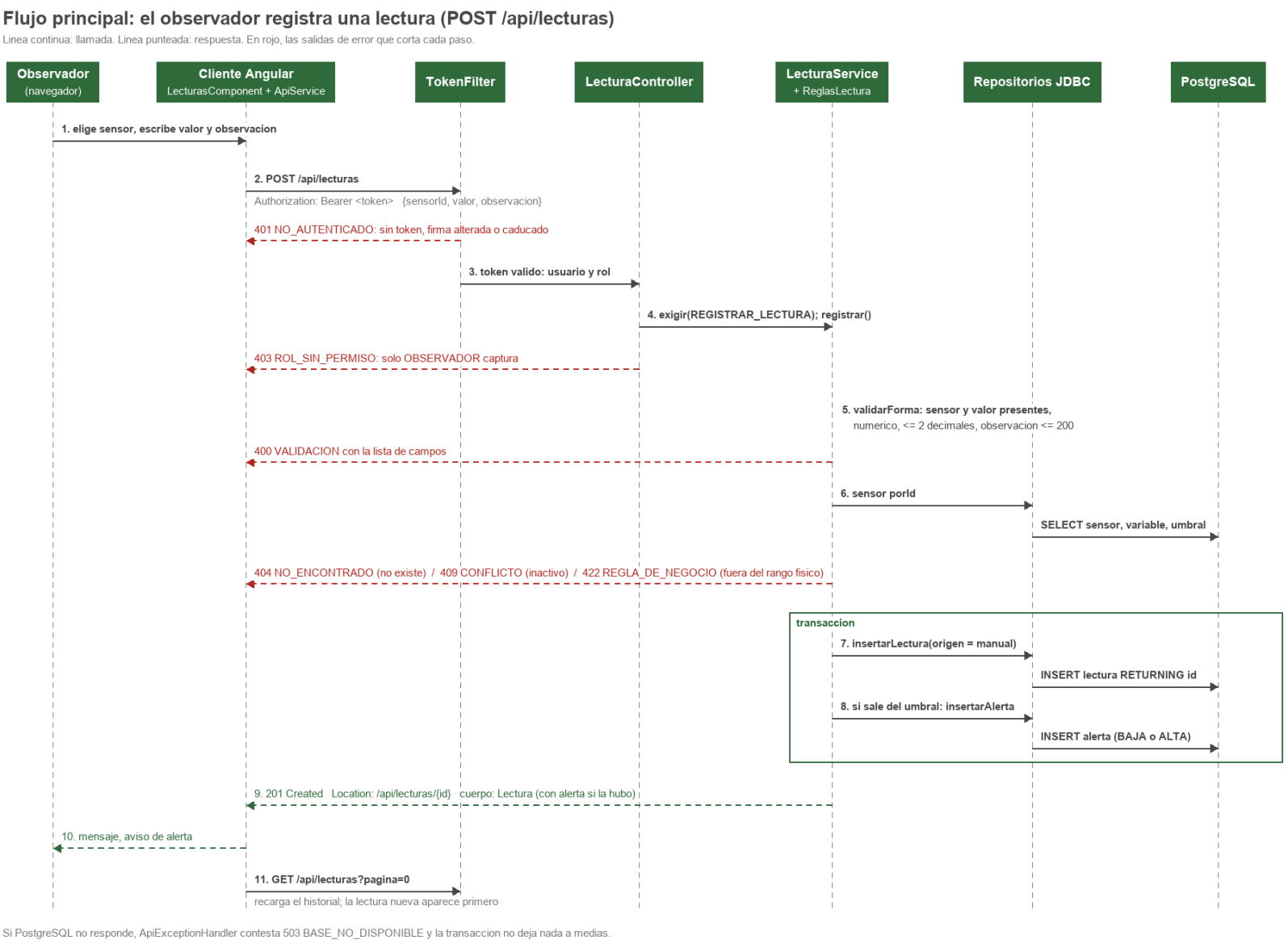


Figura 2. Secuencia de POST /api/lecturas con sus salidas de error.

El orden de las comprobaciones no es casual: primero identidad (401), luego permiso (403), después la forma de los datos (400), luego lo que requiere consultar la base (404, 409, 422) y al final la escritura. Así una petición sin permiso no llega a tocar PostgreSQL y una mal formada no abre una transacción.

5. Por qué Angular y Spring Boot: decisiones y alternativas

Angular y Spring Boot con Java 11 los fija la experiencia educativa, así que la pregunta real no era "cuál framework" sino cómo usarlos. Estas son las decisiones donde sí había alternativa, qué revisamos y qué cuesta lo que elegimos.

#	Decisión	Alternativas que revisamos	Por qué nos quedamos con esta
D1	Angular 16.2.12 con módulos (NgModule)	Componentes standalone; React o Vue; seguir con JSF	Los módulos son lo que genera el CLI 16 por omisión y lo que usa el material del curso; standalone funciona pero nos obligaba a aprender dos cosas a la vez. JSF mezcla vista y servidor, que es justo lo que P04 separa.
D2	Spring Boot 2.7.18	Spring Boot 3; servlets con JSON a mano; JAX-RS en el Tomcat de P03	Spring Boot 3 exige Java 17 y jakarta; el curso fija Java 11 y 2.7.18 es la última versión de la línea que lo soporta. Con servlets habríamos escrito a mano lo que Spring MVC ya resuelve (JSON, errores, pruebas con MockMvc).
D3	Spring JDBC (NamedParameterJdbcTemplate)	Spring Data JPA con Hibernate; JDBC puro como en P02 y P03	El SQL ya estaba escrito y probado; con Spring JDBC lo reutilizamos y vemos qué consulta se ejecuta. JPA pedía mapear siete entidades y cuidar que Hibernate no tocara un esquema que comparten tres aplicaciones. Frente al JDBC puro ganamos el pool y @Transactional. Costo: los RowMapper se escriben a mano.
D4	Token firmado (JWT HS256) sin estado en el servidor	Sesión con cookie como en P03; Spring Security con OAuth2; Basic en cada petición	Con cliente y API en orígenes distintos, la cookie obliga a resolver SameSite y CSRF; el token en la cabecera evita ambos. Spring Security es lo correcto en producción, pero para tres usuarios fijos agrega una cadena de filtros que aquí no hace falta; preferimos un filtro de 64 líneas que se lee completo. Costo: un token no se puede revocar antes de sus 30 minutos.
D5	Contrato primero: openapi.yaml a mano; springdoc como comprobación	Generar el contrato desde el código; generar el código desde el contrato	Escribirlo antes nos obligó a decidir códigos de estado y cuerpos de error sin mirar la implementación, y dejó definidos los recursos que todavía no programamos. La prueba 30 compara lo que publica el código contra el contrato. Costo: dos documentos que mantener alineados; por eso la comparación está automatizada.
D6	Un repositorio, dos carpetas	Dos repositorios; empaquetar Angular dentro del jar	Somos cuatro y la entrega es una; un repositorio mantiene juntos contrato, cliente, API y evidencia. Meter Angular en el jar simplifica el despliegue pero esconde la separación que la práctica pide mostrar.
D7	422 para valor fuera del rango físico y 400 para datos mal formados	Responder 400 a todo, como en P02	El cliente necesita distinguir "lo que enviaste no se puede leer" de "se leyó, pero el huerto no puede tener 150 °C". Son causas distintas y el mensaje también.
D8	Proxy de desarrollo (/api → 8081) y CORS de un solo origen	Llamar directo al 8081 desde el navegador; allowedOrigins("*")	Con el proxy el navegador habla con un solo origen y el cliente no lleva la URL de la API escrita. CORS queda configurado y probado para cuando se sirvan por separado. El comodín dejaría que cualquier página llamara a la API.

#	Decisión	Alternativas que revisamos	Por qué nos quedamos con esta
D9	El valor viaja como número JSON y la API lo lee como BigDecimal	Enviarlo como texto; usar double	double convierte 24.555 en una aproximación y la regla "máximo dos decimales, sin redondear" deja de ser comprobable. Con BigDecimal la escala es exacta.

Cada integrante desarrolla una de estas decisiones con más detalle en su justificación individual.

6. Modelo de datos en PostgreSQL

El modelo es el de P02 y P03: siete tablas creadas por sql/01_schema.sql. En P04 no se agregó, quitó ni modificó ninguna columna. Lo nuevo es cómo se convierte cada tabla en un recurso de la API.

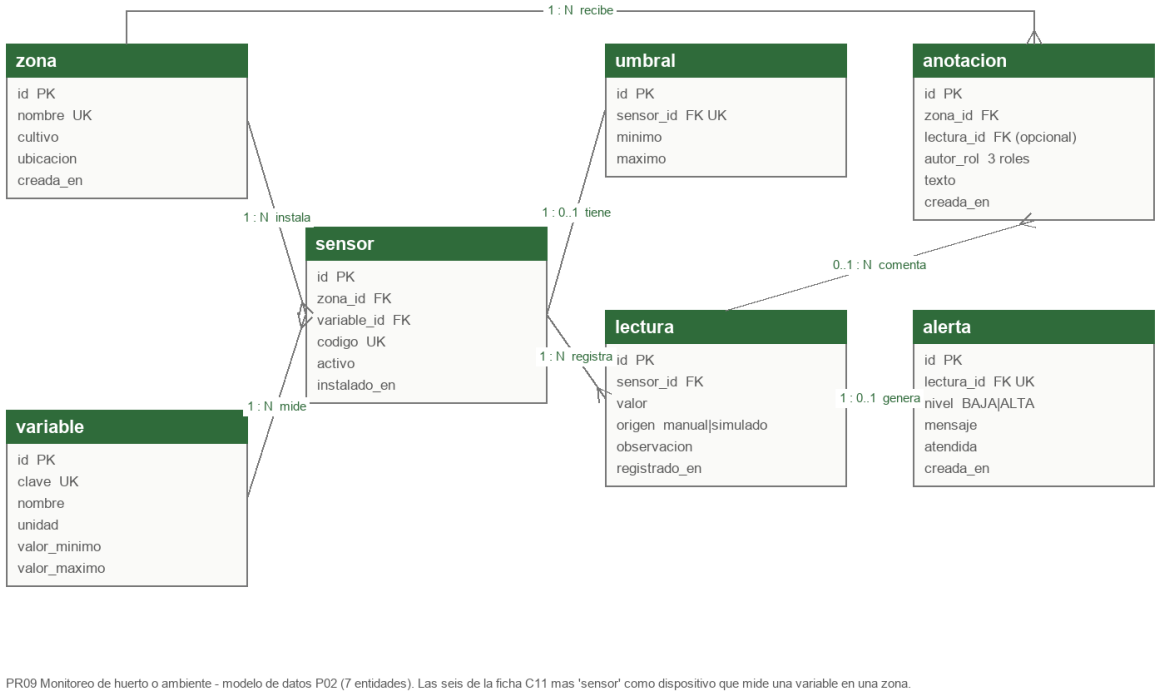


Figura 3. Modelo entidad-relación (sin cambios desde P02).

Tabla	Columnas	Restricciones
zona	id, nombre, cultivo, ubicacion, creada_en	PK; nombre único; ubicacion obligatoria
variable	id, clave, nombre, unidad, valor_minimo, valor_maximo	PK; clave única; CHECK (valor_minimo < valor_maximo)
sensor	id, zona_id, variable_id, codigo, activo, instalado_en	PK; FK a zona y variable; codigo único
umbral	id, sensor_id, minimo, maximo	PK; FK única a sensor; CHECK (minimo < maximo)
lectura	id, sensor_id, valor, origen, observacion, registrado_en	PK; FK a sensor; CHECK origen IN ('manual','simulado'); índice (sensor_id, registrado_en DESC)
alerta	id, lectura_id, nivel, mensaje, atendida, creada_en	PK; FK única a lectura; CHECK nivel IN ('BAJA','ALTA')

Tabla	Columnas	Restricciones
anotacion	id, zona_id, lectura_id, autor_rol, texto, creada_en	PK; FK a zona; FK opcional a lectura; CHECK autor_rol con los tres roles; texto mínimo 3

6.1 De tabla a recurso

La API no expone las tablas una por una. Un recurso junta lo que el cliente necesita en una sola respuesta, para que Angular no haga tres llamadas y cruce identificadores.

Recurso del contrato	Tablas	Campo JSON ← columna
Sensor	sensor + zona + variable + umbral (LEFT JOIN)	codigo ← sensor.codigo; zona ← zona.nombre; variable, unidad ← variable; rangoFisicoMinimo/Maximo ← variable.valor_minimo/valor_maximo; umbralMinimo/Maximo ← umbral
Lectura	lectura + sensor + zona + variable + alerta (LEFT JOIN)	valor, origen, observacion, registradoEn ← lectura; sensorCodigo, zona, variable, unidad ← joins; alerta{nivel, mensaje} ← alerta
NuevaLectura (entrada)	lectura	sensorId → sensor_id; valor → valor; observacion → observacion. origen no se acepta: la API escribe manual; registrado_en lo pone PostgreSQL
Zona, Alerta, Anotacion	solo contrato	directo más joins; autorRol sale del token

Convenciones: columnas en snake_case, campos JSON en camelCase; NUMERIC(10,2) viaja como número con dos decimales y en Java es BigDecimal; TIMESTAMPTZ viaja en ISO-8601 UTC y el cliente lo muestra en hora local.

6.2 Dónde vive cada regla

Regla	En la API	En PostgreSQL
Sensor obligatorio y existente	ReglasLectura, LecturaService (400, 404)	FK lectura.sensor_id
Sensor activo	LecturaService (409)	—
Máximo dos decimales	ReglasLectura (400)	NUMERIC(10,2) redondearía; por eso se valida antes
Dentro del rango físico	ReglasLectura (422)	el rango está en variable; la comparación la hace la API
Alerta si sale del umbral, límites incluidos	ReglasLectura.nivelAlerta	CHECK nivel; una alerta por lectura (UNIQUE)
Lectura y alerta juntas o ninguna	@Transactional en LecturaService	transacción
Procedencia	la API escribe manual	CHECK origen
Observación de hasta 200	ReglasLectura (400)	VARCHAR(200)

7. Endpoints y contratos

El contrato formal es docs/p04/openapi.yaml (OpenAPI 3.0.3, versión 0.4.0), validado con openapi-spec-validator. Aquí va el resumen; los ejemplos están copiados de las respuestas reales que quedaron en la evidencia.

#	Método y ruta	Para qué	Quién	Éxito	Errores	Estado
1	GET /api/health	Saber si la API llega a PostgreSQL	público	200	503	implementado
2	POST /api/auth/login	Cambiar credenciales por un token	público	200	400, 401	implementado
3	GET /api/sensores	Catálogo con rango y umbral	3 roles	200	400, 401	implementado
4	GET /api/lecturas	Historial paginado y filtrado	3 roles	200	400, 401	implementado
5	GET /api/lecturas/{id}	Una lectura	3 roles	200	401, 404	implementado
6	POST /api/lecturas	Registrar lectura manual	OBSERVADOR	201	400, 401, 403, 404, 409, 422, 503	implementado
7	GET /api/zonas	Zonas	3 roles	200	401	solo contrato
8	GET /api/alertas	Alertas	3 roles	200	400, 401	solo contrato
9	GET /api/anotaciones	Anotaciones	3 roles	200	401	solo contrato
10	POST /api/anotaciones	Registrar anotación	3 roles	201	400, 401, 404	solo contrato

7.1 Reglas comunes

Salvo las rutas 1 y 2, toda petición lleva Authorization: Bearer <token>. Sin token, con la firma alterada o caducado, la respuesta es 401 con la cabecera WWW-Authenticate. Todos los errores tienen la misma forma:

```
{
  "codigo": "VALIDACION",
  "mensaje": "La peticion tiene datos invalidos; no se guardo nada.",
  "errores": [ { "campo": "valor", "mensaje": "El valor admite como maximo 2 decimales; no se
redondea." } ],
  "instante": "2026-09-29T12:27:52.5797281-06:00"
}
```

Estado	codigo	Cuándo
400	VALIDACION	JSON mal formado, campo obligatorio ausente, tipo incorrecto, más de dos decimales, texto demasiado largo, parámetro inválido
401	NO_AUTENTICADO	Falta el token, no es válido o caducó; credenciales incorrectas
403	ROL_SIN_PERMISO	El rol del token no puede hacer la operación
404	NO_ENCONTRADO	El recurso pedido o referido no existe
409	CONFLICTO	El recurso existe pero su estado no admite la operación (sensor inactivo)
422	REGLA_DE_NEGOCIO	Datos bien formados que violan una regla del dominio (rango físico)

Estado	codigo	Cuándo
503	BASE_NO_DISPONIBLE	PostgreSQL no responde

7.2 POST /api/lecturas, el contrato que más importa

Campo de entrada	Tipo	Obligatorio	Regla
sensorId	entero	sí	debe existir y estar activo
valor	número	sí	máximo dos decimales; dentro del rango físico de la variable
observacion	texto	no	hasta 200 caracteres

No se aceptan origen ni registradoEn: la API escribe manual y PostgreSQL pone el instante. Entrada y respuesta reales de la prueba 12:

```
> POST /api/lecturas   Authorization: Bearer ...
  {"sensorId": 2, "valor": 25.5, "observacion": "[prueba] P04 lectura valida"}
< 201 Created   Location: /api/lecturas/62
  {"id":62,"sensorId":2,"sensorCodigo":"SEN-A-TEMP-01","variable":"Temperatura
ambiente","zona":"Cama A",
   "valor":25.50,"unidad":"C","origen":"manual","observacion":"[prueba] P04 lectura valida",
   "registradoEn":"2026-09-29T18:27:50.587461Z"}
```

Con 38 la respuesta también es 201, pero trae el objeto alerta con nivel ALTA y el mensaje "Temperatura ambiente en Cama A: 38 C por encima del maximo 32.00 C.". Los límites del umbral cuentan como dentro: 15 y 32 no generan alerta.

Entrada	Respuesta	Cambio en la base
valor 25.5	201 sin alerta	+1 lectura
valor 38 / 10	201 con alerta ALTA / BAJA	+1 lectura, +1 alerta
valor 15 / 32	201 sin alerta	+1 lectura
valor "abc", 24.555, sin valor, sin sensor, JSON roto, observación de 201	400 VALIDACION	ninguno
valor 150	422 REGLA_DE_NEGOCIO, "fuera del rango fisico ... [-10.00, 60.00]"	ninguno
sensorId 999	404 NO_ENCONTRADO	ninguno
sensor inactivo	409 CONFLICTO	ninguno
sin token	401 NO_AUTENTICADO	ninguno
token de COORDINACION o RESPONSABLE	403 ROL_SIN_PERMISO	ninguno

springdoc publica en /v3/api-docs el OpenAPI que deduce del código. No es el de referencia porque solo conoce lo implementado y nombra los esquemas según las clases Java; sirve como comprobación: la prueba 30 revisa que toda operación del código esté en openapi.yaml y que lo que falte sea solo lo marcado como pendiente.

8. Seguridad básica

Tema	Qué se hizo
Autenticación	Usuario y contraseña contra hash PBKDF2-HMAC-SHA256 (120 000 iteraciones, sal por usuario); comparación en tiempo constante
Token	JWT HS256 con usuario, rol y caducidad de 30 minutos. El secreto llega por JWT_SECRET; con menos de 32 caracteres la API no arranca; si falta se genera uno aleatorio por arranque
Autorización	El rol sale del token, nunca del cuerpo. POST /api/lecturas exige REGISTRAR_LECTURA (solo OBSERVADOR)
CORS	Un solo origen (CORS_ORIGIN), métodos GET, POST y OPTIONS, cabeceras Authorization, Content-Type y Accept
Inyección SQL	Todas las consultas usan parámetros con nombre
Validación	En el servidor siempre; el cliente solo ayuda
Errores	Cuerpo uniforme sin trazas ni nombres de clases; /api/health publica solo "PostgreSQL 16.15"
Secretos	application.properties solo tiene referencias ${...}; el repositorio no guarda contraseñas reales (lo revisa verify-module.sh M04)
CSRF	No aplica: el token viaja en una cabecera que el navegador no envía solo

Límites que conocemos: el token vive en sessionStorage, así que un script inyectado en la página podría leerlo (Angular escapa las salidas por omisión y no usamos innerHTML); no hay HTTPS porque todo corre en localhost; no hay límite de intentos de acceso. Los usuarios y contraseñas son ficticios, de laboratorio.

9. Despliegue y configuración

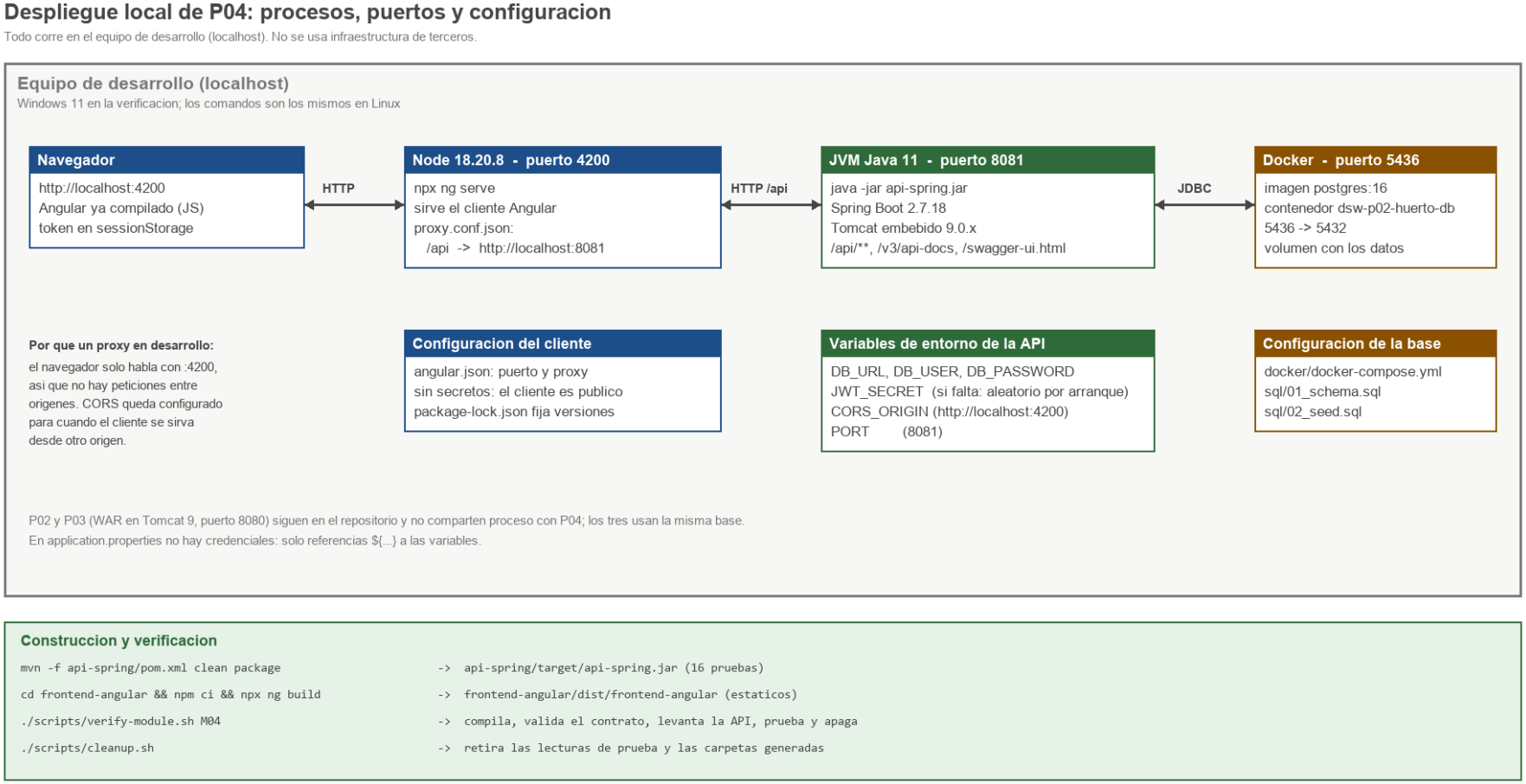


Figura 4. Despliegue local: procesos, puertos y variables.

Variable	Para qué	Valor por omisión
DB_URL	URL JDBC de PostgreSQL	ninguno; sin ella la API no arranca
DB_USER, DB_PASSWORD	Credenciales de la base	ninguno
JWT_SECRET	Secreto de firma del token	aleatorio por arranque
CORS_ORIGIN	Origen permitido del cliente	http://localhost:4200
PORT	Puerto de la API	8081

En desarrollo el navegador solo habla con el 4200: ng serve reenvía /api al 8081 (proxy.conf.json), así que no hay peticiones entre orígenes. CORS queda configurado para cuando el cliente se sirva desde otro lado.

10. La estructura compila y corre

```
dsw-p02-monitoreo-huerto/
├── api-spring/                       Spring Boot 2.7.18, Java 11
│   ├── pom.xml                       web, jdbc, postgresql 42.7.2, springdoc 1.7.0, test
│   └── src/main/java/mx/uv/dsw/huerto/api/
│       ├── config/      WebConfig
│       ├── seguridad/   TokenFilter, TokenService, CredencialesService, AuthController, Permisos
│       ├── error/       ApiError, ApiException, ApiExceptionHandler
│       ├── salud/       HealthController
│       ├── sensores/    Sensor, SensorRepository, SensorController
│       └── lecturas/    LecturaRequest, Lectura, Pagina, ReglasLectura, LecturaRepository,
│                        LecturaService, LecturaController
├── frontend-angular/                 Angular 16.2.12, Node 18.20.8
│   ├── proxy.conf.json               /api -> http://localhost:8081
│   └── src/app/
│       ├── core/        modelos.ts, api.service.ts, sesion.service.ts, token.interceptor.ts,
sesion.guard.ts
│       ├── acceso/      acceso.component
│       └── lecturas/    lecturas.component
├── docs/p04/                         openapi.yaml, arquitectura, modelo, endpoints, trazabilidad,
verificación
└── scripts/                          verify-m04.sh, pruebas_api.py, recorrido_angular.py,
prueba_base_detenida.sh
```

Resultado de la última compilación, la del 9 de octubre (evidencia 01_build_api.txt y 05_build_frontend.txt):

```
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
| Initial Total | 334.77 kB |                89.13 kB
Build at: 2026-10-10T03:57:23.077Z - Hash: 5c02722d8d2781ea - Time: 1719ms
```

Las 16 pruebas del backend cubren las reglas (ReglasLecturaTest), el token (TokenServiceTest: emisión, caducidad, firma alterada, secreto corto) y el controlador con MockMvc (LecturaControllerTest: 401, 200, 201 con Location, 403, 400, 404, 409, 422 y 503). Las 5 del cliente cubren la sesión por rol, la sesión caducada y el cuerpo que envía ApiService.

11. Trazabilidad RF → componente

RF	Requisito	Operación	Componentes (cliente / API / base)	Prueba	Estado
RF01	Zonas	GET /api/zonas; la zona viaja dentro de Sensor y Lectura	columna Zona / repositorios (join con zona) / zona	contrato 07, 09	dentro de sensores y lecturas: VERIFICADO; /api/zonas: PENDIENTE
RF02	Variables con unidad y rango físico	GET /api/sensores	ficha del sensor / SensorController, SensorRepository / variable	contrato 07; navegador 03	VERIFICADO
RF03	Registrar lecturas	POST /api/lecturas	LecturasComponent, ApiService / LecturaController, LecturaService, ReglasLectura, LecturaRepository / lectura	contrato 12, 17 a 26; navegador 04, 06, 07; JUnit	VERIFICADO
RF04	Umbrales	GET /api/sensores (umbralMinimo, umbralMaximo)	ficha del sensor / SensorRepository, ReglasLectura.nivelAlerta / umbral	contrato 07, 16; JUnit	VERIFICADO
RF05	Historial	GET /api/lecturas, GET /api/lecturas/{id}	tabla, filtro y paginador / LecturaService.listar, LecturaRepository / lectura e índice	contrato 09 a 11, 13, 28; navegador 08	VERIFICADO
RF06a	Alerta automática	objeto alerta en POST /api/lecturas; GET /api/alertas	aviso y etiqueta / LecturaService (transacción), insertarAlerta / alerta	contrato 14 a 16; navegador 05	generación: VERIFICADO; /api/alertas: PENDIENTE
RF06b	Anotaciones por rol	GET y POST /api/anotaciones	— / — / anotacion (ya existe)	—	PENDIENTE
RF07	Modelo relacionado	todas	— / repositorios / 7 tablas con FK, UNIQUE y CHECK	03_consultas_verificacion.sql antes y después	VERIFICADO
RF08	API Web 3.0	openapi.yaml completo	ApiService, TokenInterceptor / api-spring/ / —	verify-module.sh M04	6 de 10 operaciones VERIFICADO; 4 PENDIENTE
RF09	Fuente IoT simulada	origen=simulado en GET /api/lecturas	filtro de procedencia / LecturaRepository / lectura.origen	contrato 10; navegador 08	consulta VERIFICADO; el simulador no es parte de P04
RNF01	Ruta de salud	GET /api/health	— / HealthController / —	contrato 01; base detenida	VERIFICADO (200 y 503)
RNF02	Autorización por rol	todas	sesionGuard, TokenInterceptor / TokenFilter, Permisos, UsuarioAutenticado / —	contrato 05, 06, 26, 27; navegador 01, 10, 11	VERIFICADO

La matriz completa, con la lectura inversa (de componente a requisito) y la continuidad P02 → P03 → P04, está en docs/p04/trazabilidad-rf.md.

12. Pruebas y evidencia

Todo se regenera con un comando: ./scripts/verify-module.sh M04 compila la API y el cliente, corre sus pruebas, valida openapi.yaml, arranca la API, ejecuta los 30 casos del contrato, sirve el cliente, hace el recorrido en navegador, consulta PostgreSQL antes y después, y apaga todo. Lo corrimos completo el 29 de septiembre, cuando cerramos el código, y otra vez el 9 de octubre antes de entregar; las dos veces terminó con RESULTADO: VERIFICADO, sin fallos. Las salidas que siguen son de la corrida final.

```
verify-module.sh M04 - 2026-10-09T21:58:07-06:00 - commit bf78beb
VERIFICADO    | Java 11 disponible
VERIFICADO    | Node 18 disponible
VERIFICADO    | api-spring compila con Java 11 y sus pruebas pasan
VERIFICADO    | El jar contiene controladores, driver de PostgreSQL, Spring JDBC y springdoc
VERIFICADO    | application.properties toma conexion y secreto de variables de entorno
VERIFICADO    | frontend-angular compila y genera dist/frontend-angular
VERIFICADO    | Las pruebas del cliente Angular pasan
VERIFICADO    | openapi.yaml es OpenAPI 3.0 valido
VERIFICADO    | PostgreSQL responde y el modelo de 7 tablas existe
VERIFICADO    | La API arranca y /api/health responde 200
VERIFICADO    | /api/health reporta db=UP
VERIFICADO    | pruebas_api.py termino sin casos NO_VERIFICADO
VERIFICADO    | recorrido_angular.py termino sin casos NO_VERIFICADO
VERIFICADO    | Las lecturas creadas por la API existen en PostgreSQL
VERIFICADO    | La API se detuvo y el puerto 8081 quedo libre
Fallos: 0
```

12.1 Contrato contra la API en marcha (30 casos)

Cada caso no solo revisa el código de estado: valida el cuerpo contra el esquema del contrato y, en los casos negativos, cuenta filas en PostgreSQL antes y después para comprobar que no se guardó nada.

```
pruebas_api.py - 2026-10-09T21:57:45 - http://localhost:8081
Casos: 30  VERIFICADO: 30  NO_VERIFICADO: 0

01 | VERIFICADO    | Salud publica: 200, base UP
02 | VERIFICADO    | Login correcto: 200 con token y rol OBSERVADOR
03 | VERIFICADO    | Login incorrecto: 401 sin token
04 | VERIFICADO    | Login incompleto: 400
05 | VERIFICADO    | Recurso protegido sin token: 401
06 | VERIFICADO    | Token con firma alterada: 401
07 | VERIFICADO    | Catalogo de sensores: 200 con rango fisico y umbral
08 | VERIFICADO    | Parametro de consulta no numerico: 400
09 | VERIFICADO    | Historial paginado: 200 y total igual al de PostgreSQL
10 | VERIFICADO    | Filtro por procedencia: solo lecturas simuladas
11 | VERIFICADO    | Procedencia desconocida: 400
12 | VERIFICADO    | Lectura valida: 201, Location, procedencia manual, sin alerta, +1 lectura
13 | VERIFICADO    | La lectura creada se recupera por su identificador
14 | VERIFICADO    | Lectura sobre el umbral: 201 con alerta ALTA, +1 lectura y +1 alerta
15 | VERIFICADO    | Lectura bajo el umbral: 201 con alerta BAJA
16 | VERIFICADO    | Limites inclusivos: 15 y 32 se guardan sin alerta
17 | VERIFICADO    | Rechazo: valor no numerico "abc" -> 400 VALIDACION, la base no cambia
18 | VERIFICADO    | Rechazo: valor con tres decimales 24.555 -> 400 VALIDACION, la base no cambia
19 | VERIFICADO    | Rechazo: sin valor -> 400 VALIDACION, la base no cambia
20 | VERIFICADO    | Rechazo: sin sensor -> 400 VALIDACION, la base no cambia
21 | VERIFICADO    | Rechazo: JSON mal formado -> 400 VALIDACION, la base no cambia
22 | VERIFICADO    | Rechazo: observacion de 201 caracteres -> 400 VALIDACION, la base no cambia
```

```
23 | VERIFICADO    | Rechazo: valor 150, fuera del rango fisico -> 422 REGLA_DE_NEGOCIO, la base no
cambia
24 | VERIFICADO    | Rechazo: sensor 999, no existe -> 404 NO_ENCONTRADO, la base no cambia
25 | VERIFICADO    | Sensor inactivo: 409 y la base no cambia
26 | VERIFICADO    | Registrar sin token: 401 y la base no cambia
27 | VERIFICADO    | Rol sin permiso de captura: 403 al registrar, 200 al consultar
28 | VERIFICADO    | Lectura inexistente: 404
29 | VERIFICADO    | CORS: solo se admite el origen del cliente Angular
30 | VERIFICADO    | Todo lo implementado esta en el contrato; lo pendiente son zonas, alertas y
anotaciones
```

12.2 Recorrido del cliente en navegador (12 casos)

Las capturas se tomaron con Playwright contra el cliente servido por ng serve, que reenvía /api a la API real.

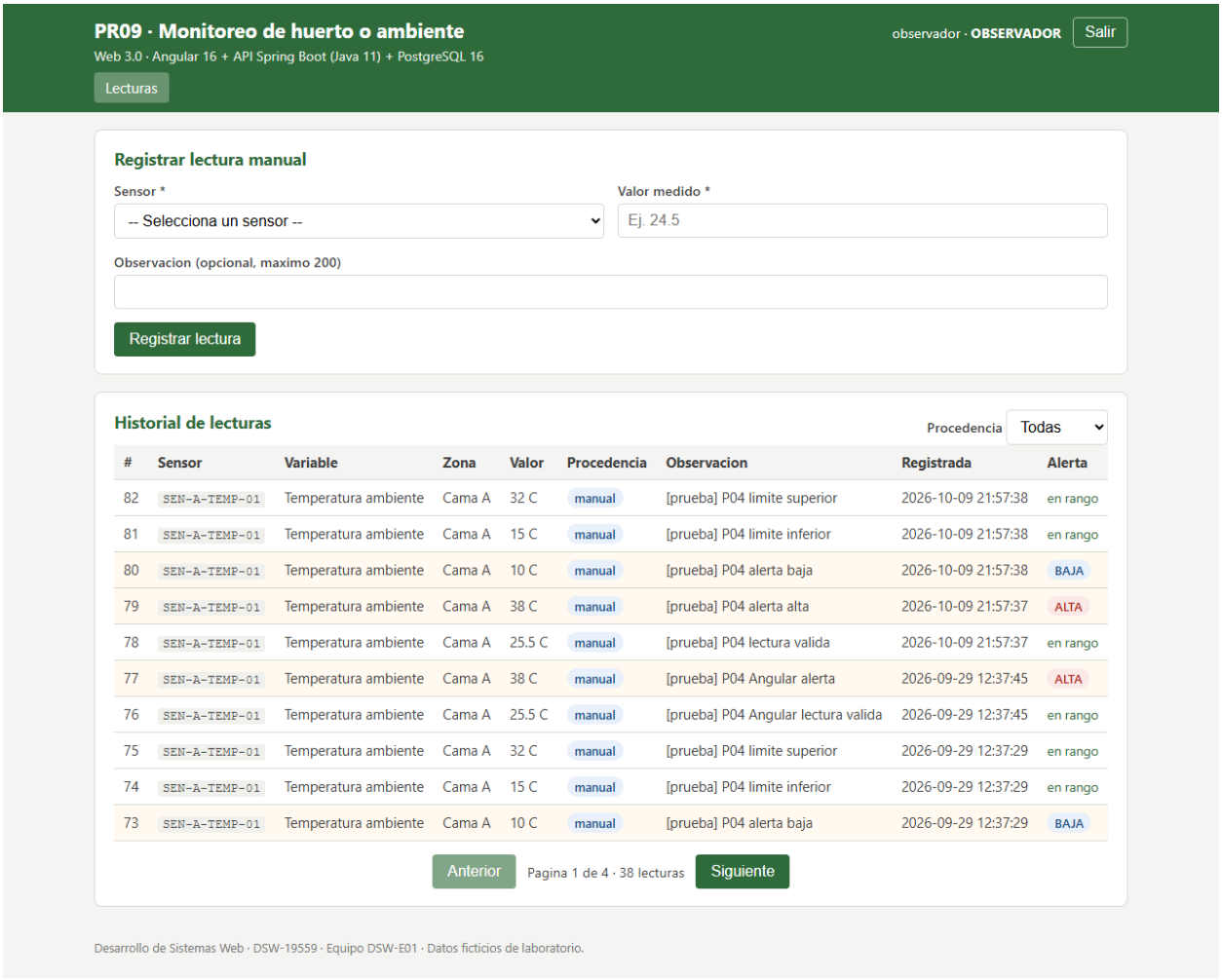


Figura 5. El observador entra: formulario de captura e historial que llegan de la API.

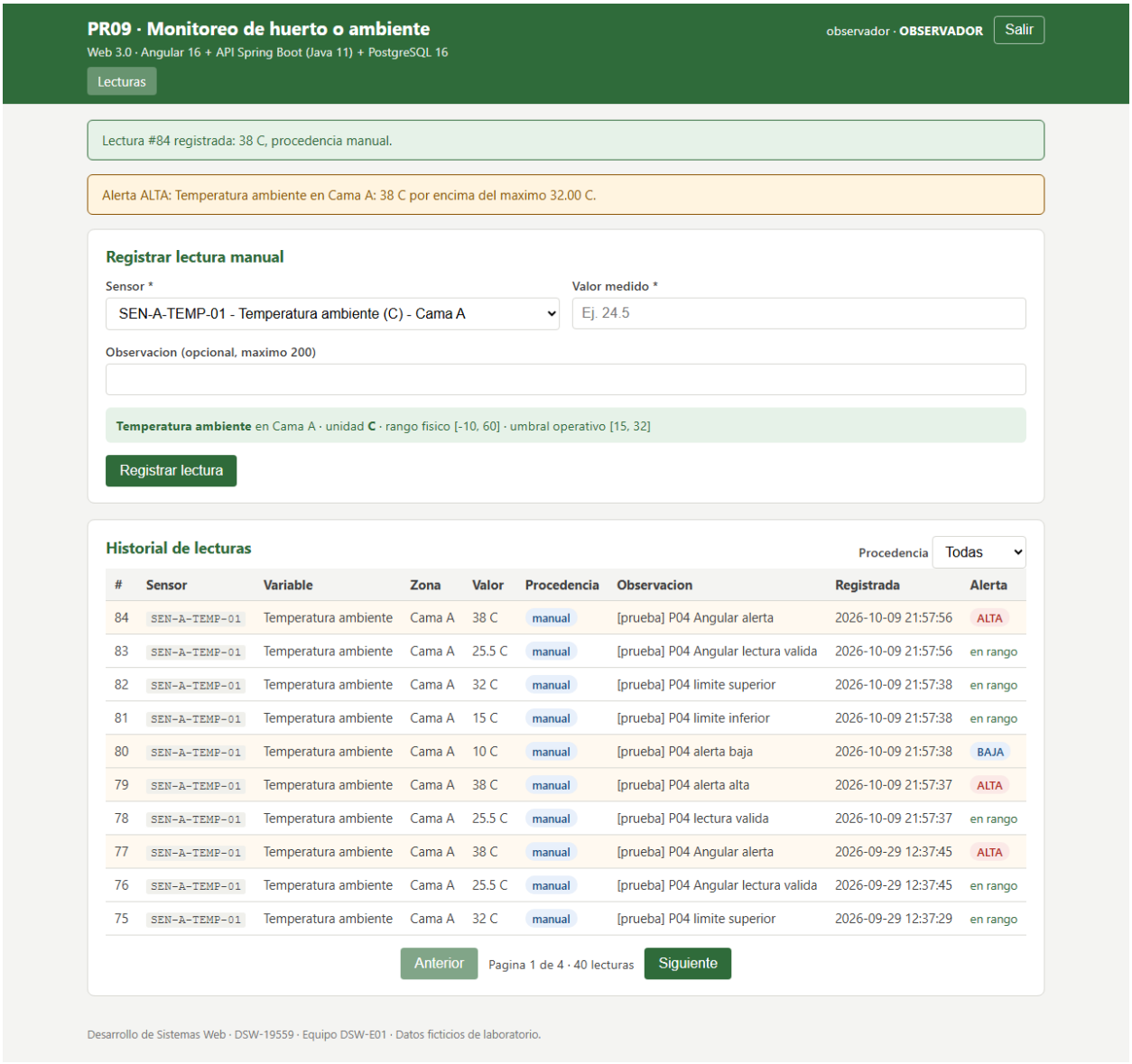


Figura 6. Lectura de 38 °C: la API responde 201 con alerta ALTA y el cliente la muestra.

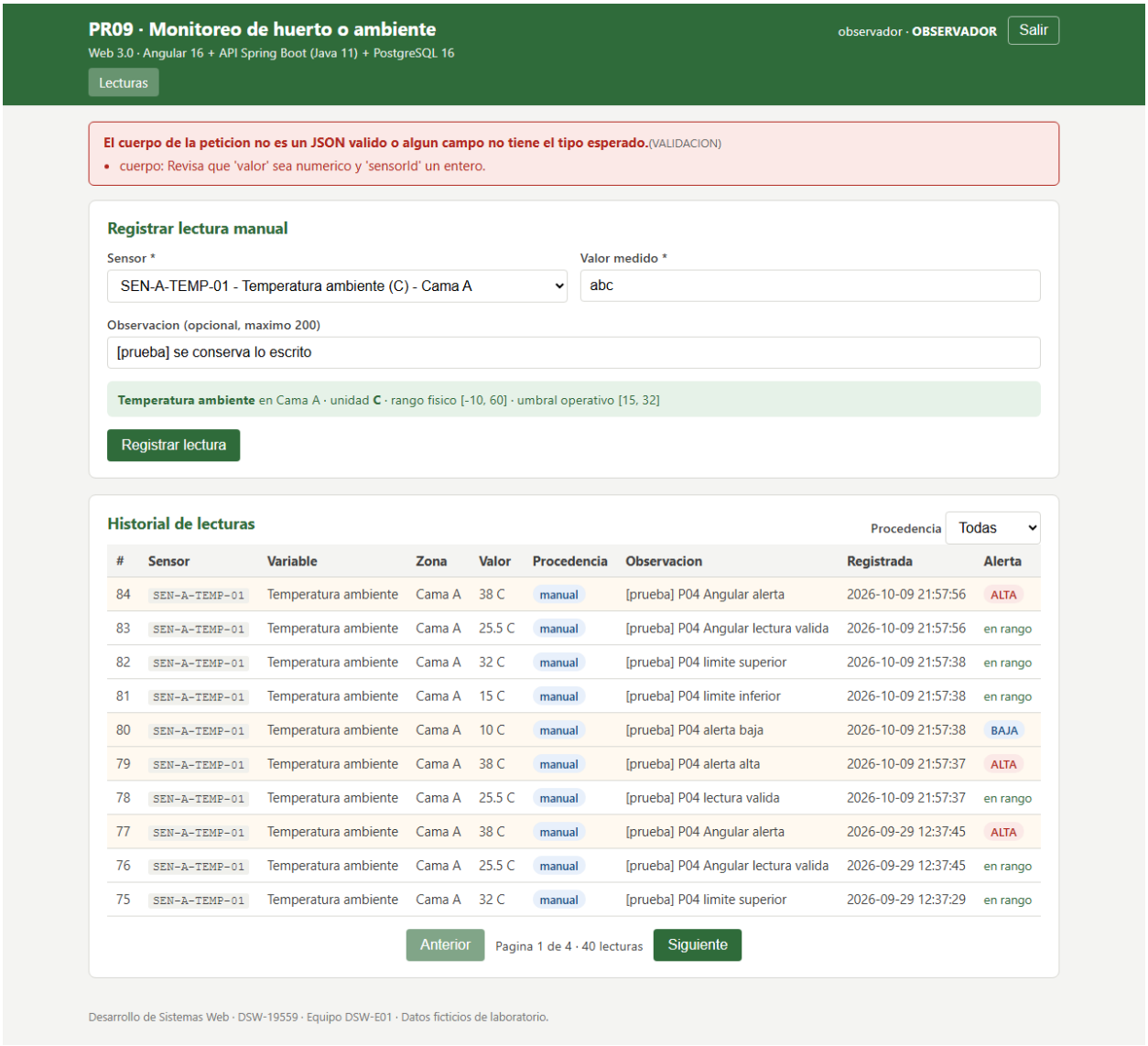


Figura 7. Valor "abc": 400 con el error por campo; lo escrito se conserva.

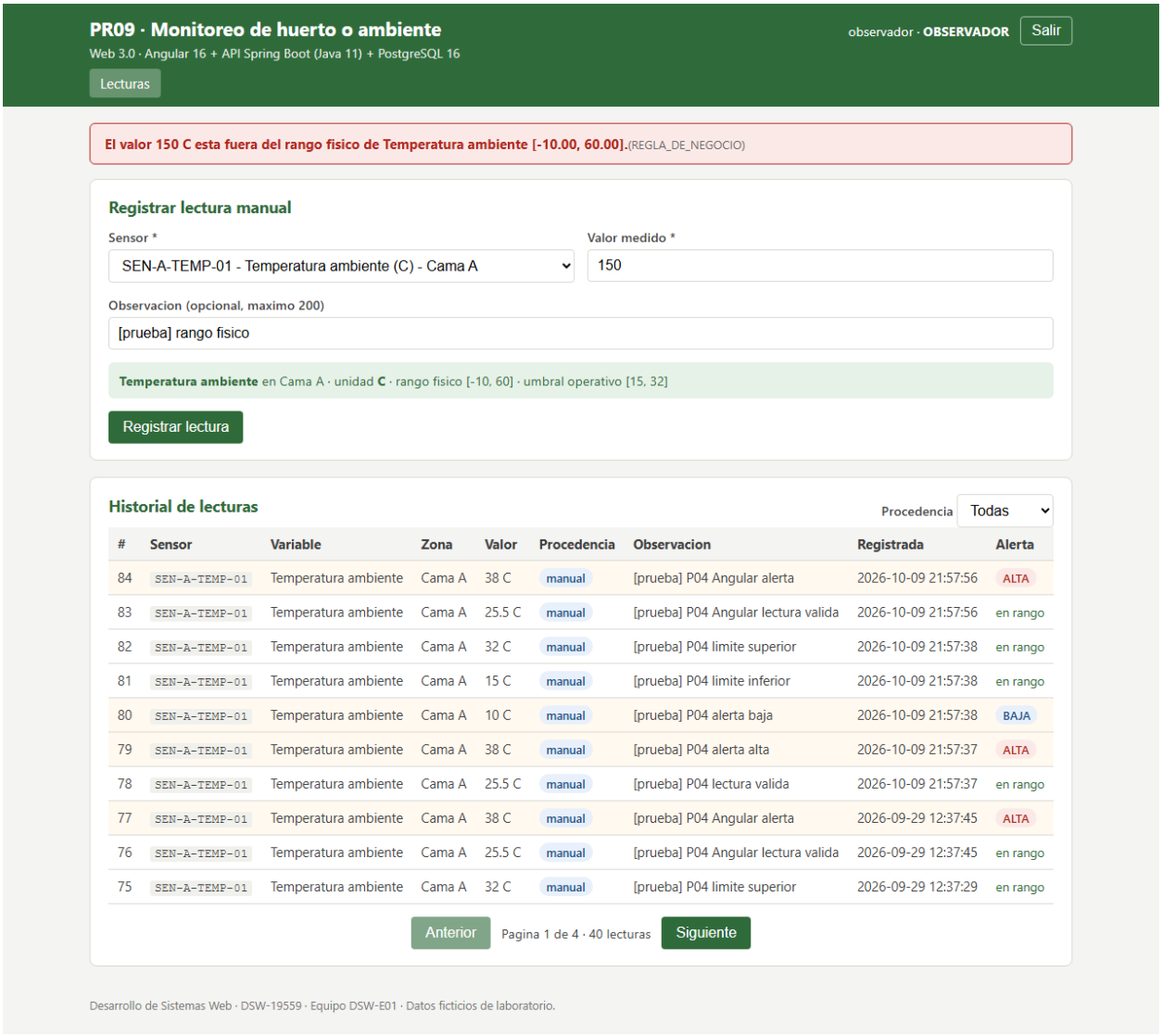


Figura 8. Valor 150: 422 con el rango físico; la base no cambia.

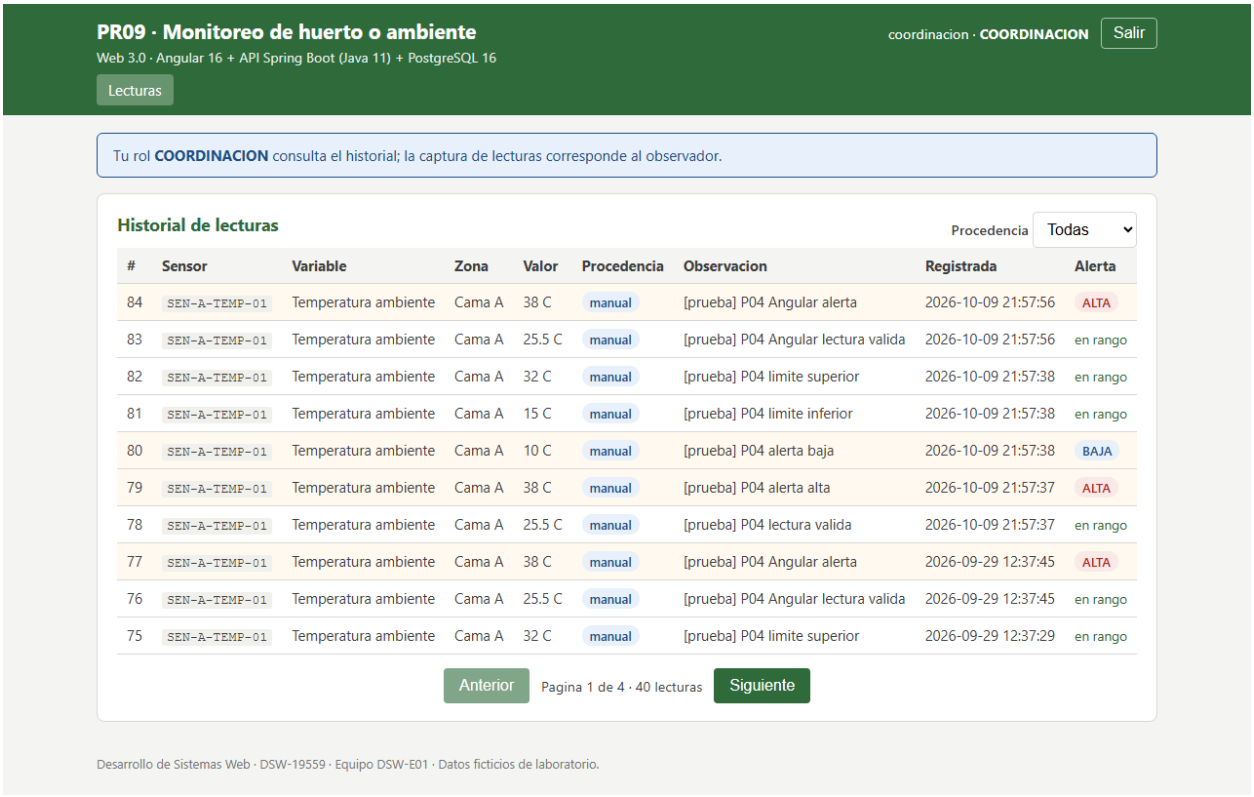


Figura 9. Coordinación consulta el historial pero no tiene formulario de captura.



Figura 10. Token alterado en el navegador: la API responde 401 y el cliente regresa al acceso.

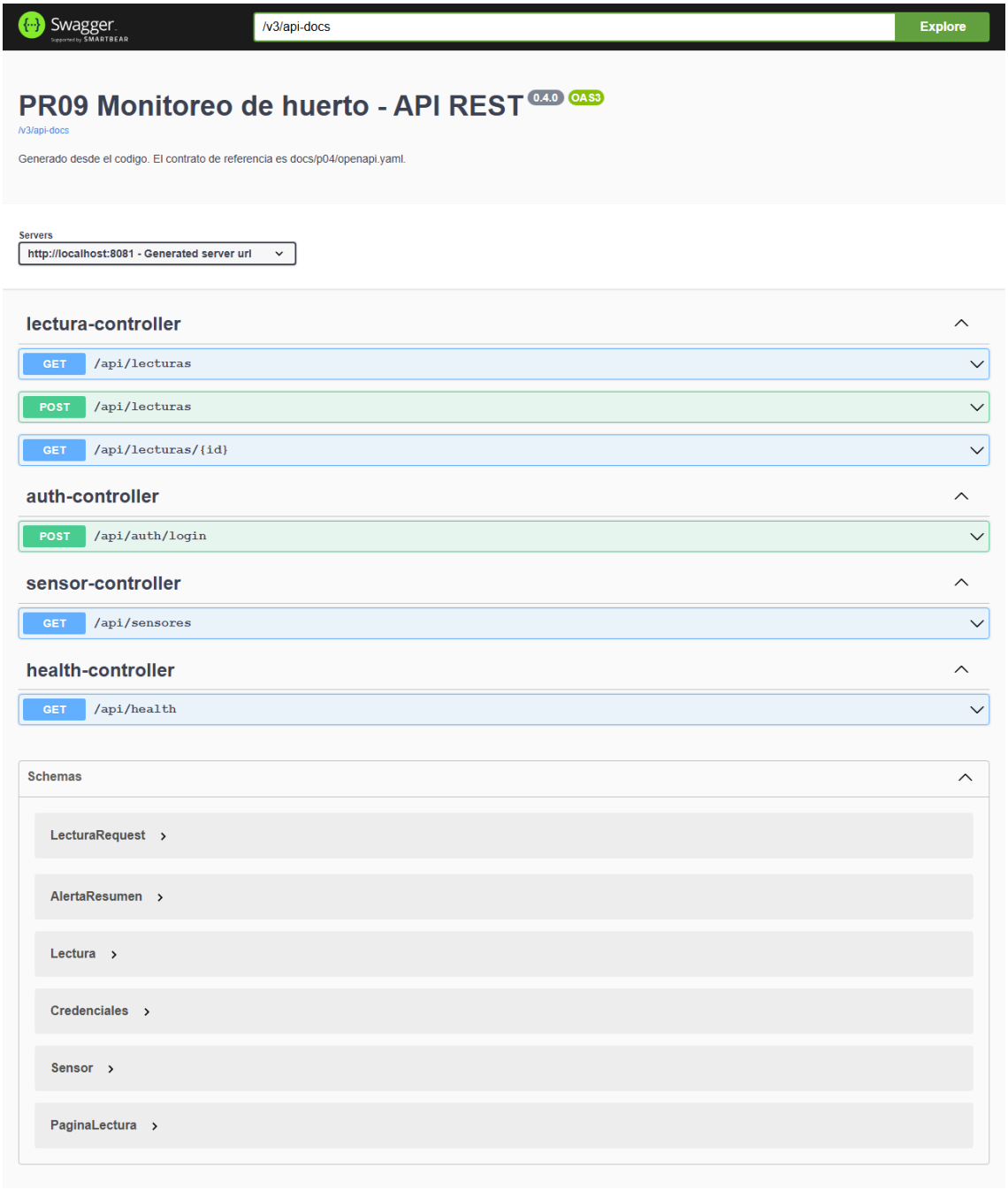


Figura 11. Operaciones que springdoc deduce del código, en /swagger-ui.html.

12.3 Base de datos detenida

Con docker stop sobre el contenedor del proyecto (sin borrar nada), la salud responde 503, la consulta y el registro responden 503 BASE_NO_DISPONIBLE, el login sigue funcionando porque no usa la base, y al volver el contenedor la API se recupera sin reiniciarla. No quedó ninguna lectura a medias.

```
--- GET /api/health
{"app":"huerto-api PR09 (P04 Spring Boot)","status":"UP","db":"UP","postgres":"PostgreSQL
16.15","lecturas":33}
[estado 200, 0.005914 s]
--- login de un usuario que no existe
```

```
{"codigo":"NO_AUTENTICADO","mensaje":"Usuario o contrasena incorrectos.","instante":"2026-09-
29T12:42:19.4649254-06:00"}
[estado 401, 0.030221 s]
--- login de observador con contrasena incorrecta
{"codigo":"NO_AUTENTICADO","mensaje":"Usuario o contrasena incorrectos.","instante":"2026-09-
29T12:42:19.5169765-06:00"}
[estado 401, 0.021950 s]
--- GET /api/health
{"app":"huerto-api PR09 (P04 Spring Boot)","status":"DOWN","db":"DOWN","error":"PostgreSQL no responde"}
[estado 503, 5.014689 s]
--- GET /api/sensores con token
{"codigo":"BASE_NO_DISPONIBLE","mensaje":"No fue posible completar la operacion en PostgreSQL; no se
guardo nada.","instante":"2026-09-29T12:42:30.0616608-06:00"}
[estado 503, 5.018525 s]
--- POST /api/lecturas con token
{"codigo":"BASE_NO_DISPONIBLE","mensaje":"No fue posible completar la operacion en PostgreSQL; no se
guardo nada.","instante":"2026-09-29T12:42:35.114822-06:00"}
[estado 503, 5.016927 s]
--- POST /api/auth/login (no usa la base)
[estado 200, 0.023324 s]
--- GET /api/health (la API se recupera sin reiniciarla)
{"app":"huerto-api PR09 (P04 Spring Boot)","status":"UP","db":"UP","postgres":"PostgreSQL
16.15","lecturas":33}
[estado 200, 0.004511 s]
lecturas con la observacion de esta prueba en PostgreSQL: 0
```

13. Problemas que tuvimos

Qué pasó	Cómo apareció	Qué hicimos
El login sin contraseña respondía 401 y el contrato dice 400	caso 04 de pruebas_api.py	AuthController valida primero que vengan los dos campos
Con la base detenida, POST /api/lecturas respondía 500 en lugar de 503	prueba_base_detenida.sh	Con @Transactional, si no hay conexión la excepción es CannotCreateTransactionException, que no es DataAccessException. El manejador atiende también TransactionException y se agregó la prueba baseNoDisponible
/api/health publicaba la versión completa de PostgreSQL, con sistema operativo y compilador	revisión de la respuesta	Se recorta a "PostgreSQL 16.15"
El registro de arranque de Spring Boot escribía el nombre del equipo y el usuario del sistema en la evidencia	revisión de 09_api_arranque.txt	spring.main.log-startup-info=false y el anonimizador sustituye ambos datos
Las pruebas de Angular no arrancaban: el navegador no se conectaba a Karma	ng test agotaba el tiempo tres veces	Chromium reciente separa el modo sin ventana en chrome-headless-shell; el script usa ese ejecutable
Un caso del recorrido fallaba por la propia prueba	caso 03 de recorrido_angular.py	Buscaba una URL que terminara en /api/sensores y el cliente envía ?activo=true

Los seis están en docs/bitacora.md con hora, comando y resultado tal cual salió.

14. Relación con la rúbrica R04

Criterio	Pts	Dónde está
Diagrama de arquitectura	3	Figuras 1 a 3 (componentes, secuencia, despliegue) y sección 3; docs/p04/arquitectura.md
Justificación Angular/Spring Boot (decisiones y alternativas)	2.5	Sección 5, decisiones D1 a D9; versiones en 00_versiones.txt
Modelo PostgreSQL	2	Sección 6; docs/p04/modelo-datos.md; sql/01_schema.sql; 08_postgres_antes.txt y 12_postgres_despues.txt
Endpoints y contratos	2.5	Sección 7; docs/p04/openapi.yaml (10 operaciones, 16 esquemas); 30 casos en pruebas_api.txt
Aportación individual (bitácora, commits o explicación propia)	2	P04_APELLIDO_NOMBRE de cada integrante; docs/bitacora.md entradas 25 a 33; git log

El índice que relaciona cada archivo de evidencia con su criterio está en docs/p04/evidencia/INDICE.md.

15. Aportación individual

La justificación individual se entrega aparte, con el nombre P04_APELLIDO_NOMBRE. Cada integrante escribe la suya: qué aportó, con el lugar del repositorio donde se ve, y una decisión técnica justificada con al menos una alternativa. Nadie escribe el registro de otra persona. Los commits de P04 en el repositorio están a nombre de la cuenta que hizo la integración (zywoxxx). El reparto acordado en el acta sigue siendo el mismo: desarrollo web (Juan Pablo Kuri Ricardez), datos (Pedro García Padilla), evidencia y documentación (Alejandro Pacheco Luna y Ariadna Trejo Álvarez).

16. Limitaciones y pendientes

Asunto	Estado	Nota
GET /api/zonas, GET /api/alertas, GET y POST /api/anotaciones y sus vistas	PENDIENTE	Definidos en el contrato; sin código
Caducidad real del token a los 30 minutos	NO_VERIFICADO	Probada con reloj fijo en JUnit; no se esperó media hora
Rol RESPONSABLE contra la API	NO_VERIFICADO	Se probó COORDINACION; ambos tienen los mismos permisos
Petición CORS real desde otro origen en el navegador	NO_VERIFICADO	Se probó el preflight con peticiones directas
Servir el resultado de ng build; despliegue en Linux	NO_VERIFICADO	Solo ng serve en Windows 11
Reproducción por otro integrante en máquina limpia	PENDIENTE	CA05
Confirmación del facilitador en F02	PENDIENTE	No se afirma que esté confirmada

Otras limitaciones: un token emitido no se puede revocar; se pierde al cerrar la pestaña; no hay límite de intentos; los usuarios están en un archivo dentro del jar; springdoc nombra los esquemas según las clases Java, así que se comparan rutas y métodos, no nombres; las lecturas de prueba quedan en la base hasta correr cleanup.sh.

Anexo. Comandos

```
docker compose -f docker/docker-compose.yml up -d        # la misma base de P02 y P03
cd frontend-angular && npm ci && cd ..                   # una vez
python -m pip install pyyaml openapi-spec-validator jsonschema playwright && python -m playwright
install chromium

# API (terminal 1)
export DB_URL=jdbc:postgresql://localhost:5436/huerto_db DB_USER=huerto_app DB_PASSWORD=huerto_dev
mvn -f api-spring/pom.xml clean package && java -jar api-spring/target/api-spring.jar

# Cliente (terminal 2)
cd frontend-angular && npx ng serve                      # http://localhost:4200

# Verificación y limpieza (con la API y el cliente detenidos)
./scripts/verify-module.sh M04
./scripts/prueba_base_detenida.sh
./scripts/cleanup.sh
```